

UNIVERSIDADE FEDERAL DE JUIZ DE FORA
CYGNO - Grupo UFJF

Atividade: Projeto de um classificador

Introdução e Objetivos

Após a familiarização com os dados, o objetivo agora é construir, treinar e avaliar um modelo de Machine Learning capaz de diferenciar eventos de Recuo Eletrônico (ER) e Recuo Nuclear (NR).

A maior parte das funções que precisarão ser utilizadas estão disponíveis na biblioteca `scikit-learn`.

Ferramentas recomendadas: Google Colab ou VSCode com ambiente Python configurado.

Parte 1: Análise dos Dados

Nesta primeira parte, vamos carregar o Dataset e visualizar suas características através de histogramas, semelhante ao que foi feito na atividade introdutória.

Carregamento e Visualização do Dataset

- **Tarefa:** Carregue o arquivo `dataset.csv`.

  Dica: use a biblioteca `pandas`.

- **Tarefa:** Inspecione o DataFrame para entender a estrutura do Dataset.

  Dica: use funções do `pandas`, como `.head()`, `.info()`, `.describe()`, entre outras que estão disponíveis.

Análise por Histogramas

- **Tarefa:** Para cada feature, plote um histograma comparando a distribuição dos valores para as classes "ER", "He", "F", "C" e "Noise". Faça isso para cada nível de energia. Use a opção 'density=True' no histograma para comparar as formas das distribuições.

  "He", "F", "C' são tipos de eventos de NR (Hélio, Fluor e Carbono).

- **Pergunta:** A partir da comparação dos histogramas, quais conclusões podem ser tiradas sobre a capacidade de discriminação entre as classes? Em particular, em quais níveis de energia a separação entre ER e NR parece mais evidente, e quais variáveis apresentam maior poder discriminatório?

Parte 2: Construção do Classificador

Agora vamos ao Machine Learning. Para esta atividade, projetaremos um classificador simples, sem otimização e sem pré-processamento de dados. Focaremos em um problema de **classificação binária**: distinguir apenas ER de NR. O "Noise" não será utilizado aqui. Para representar o NR, escolha um dentre "He", "F", "C'.

- **Pergunta:** Após visualizar os histogramas, você identifica algum critério para fazer a escolha do tipo de NR?

Preparação dos Dados para o Modelo

Para um modelo de Machine Learning, a primeira tarefa é converter o DataFrame em uma estrutura que o modelo possa entender. Esse processo tem duas partes principais:

- **Matriz de Features (X):** Representa as "características". É uma matriz (ou tabela) onde cada linha corresponde a um evento (ou cluster) e cada coluna corresponde às características que extraímos (como 'sc_integral', 'sc_length', etc.). O modelo usará os padrões encontrados em `X` para fazer suas previsões.
- **Vetor Alvo (y):** Representa o "rótulo" ou "alvo". É um vetor (uma única coluna) que contém a resposta correta para cada linha da matriz `X`. No nosso caso, o rótulo indica se o evento é um Recuo Eletrônico (ER) ou um Recuo Nuclear (NR). É isso que queremos que o nosso modelo aprenda a prever.

Vamos selecionar nossas features (X) e nosso alvo (y), e converter as classes de texto ('ER', 'NR') para números (0, 1).

Exemplo de como você pode fazer isso:

```
from sklearn.preprocessing import LabelEncoder

# Filtra o DataFrame para conter apenas ER e NR (aqui usando He por exemplo)
df_binary = df[df['particle_type'].isin(['ER', 'He'])].copy()

# Seleciona as colunas que serão usadas como features para o modelo (aqui no
↪  exemplo selecionando só uma feature)
```

```
features = [
    'sc_integral'
]
X = df_binary[features]

# Seleciona a coluna alvo (o que queremos prever)
y_text = df_binary['particle_type']

# Converte os labels de texto ('ER', 'He') para números (0, 1)
le = LabelEncoder()
y = le.fit_transform(y_text)

# Imprime para ver a conversão
print("Classes originais:", le.classes_)
print("Labels em texto:", y_text.values)
print("Labels codificados:", y)
```

Separação em Conjuntos de Treino e Teste

É fundamental separar nossos dados. Usaremos uma parte para treinar o modelo e outra (que o modelo nunca viu) para testar sua performance real.

- **Tarefa:** Use a função `train_test_split` para dividir `X` e `y` em conjuntos de treino e teste. Use 30% dos dados para teste e defina um `random_state` (é um parâmetro da função) para reprodutibilidade.

Treinamento do Modelo

- **Tarefa:** Escolha um classificador simples do Scikit-learn, como o `RandomForestClassifier`. Instancie e treine o modelo usando os dados de treino ("X_train", "y_train").

Parte 3: Avaliação de Performance

Com o modelo treinado, vamos avaliar o quão bem ele se sai ao prever os dados do conjunto de teste.

Métricas de Classificação

- **Tarefa:** Use o modelo treinado para fazer previsões no conjunto de teste (`X_test`).
- **Tarefa:** Gere e analise o `classification_report` e a `confusion_matrix` para entender a performance do modelo.

Curva ROC e AUC

A curva ROC é uma ferramenta visual para avaliar classificadores binários. Ela mostra a performance do modelo em todos os limiares de classificação.

- **Tarefa:** Calcule e plote a curva ROC para o seu modelo. Calcule também o valor da AUC (Área Sob a Curva).

Parte 4: Apresentação dos Resultados

- **Tarefa:** Prepare uma apresentação de slides para mostrar os resultados encontrados em cada etapa.